International Journal of Research Publication and Reviews

Journal homepage: www.ijrpr.com ISSN 2582-7421

Environmental Sound Classification using Audio Spectrogram Transformers (AST): A Comparative Study with CNNs

Rohan G. Rathod[1], Dr. Manjiri B. Patwari[2]

[1]Student, Master of Computer Applications, Dr. D.Y. Patil Institute of Management & Research, Pimpri, Pune 411018, Maharashtra, India
[2]Head of Research Department, Dr. D.Y. Patil Institute of Management & Research, Pimpri, Pune 411018, Maharashtra, India

ABSTRACT:

Figuring out how to make machines understand background noise technically known as Environmental Sound Classification (ESC) is a huge area in AI today, especially for building smart cities and automated security systems. In this research, we take a direct look at two major deep learning approaches to see how they stack up against each other: Convolutional Neural Networks (CNNs) and the Audio Spectrogram Transformer (AST). We trained and tested both models using the UrbanSound8K dataset under the exact same setup to find out which one actually performs better in a practical scenario. The results were clear: the lightweight CNN easily outperformed the heavier AST model on this medium-sized dataset. The CNN reached a validation accuracy of 70.4% and a Macro F1-score of 67.6%, while the AST only managed 58.2%. On top of that, the CNN processed data much faster and took up barely any storage (just 0.49 MB compared to the AST's massive 328.81 MB). Ultimately, this project proves that while Transformer models are great for massive amounts of data, a well-built CNN is still the smartest, most efficient choice for standard environmental sound recognition.

Keywords: Environmental Sound Classification, Convolutional Neural Networks, Audio Spectrogram Transformer, Deep Learning, UrbanSound8K.

1. Introduction

Sound gives us constant clues about what is happening around us. Whether it is a car honking in traffic or a machine breaking down in a factory, these audio signals carry real-time data. Getting computers to recognize these specific noises is called Environmental Sound Classification (ESC). It is a massive deal right now for building smart cities and automated security networks (Piczak, 2015). If a city microphone picks up the sound of glass breaking or a siren, the system can instantly alert the police (Salamon & Bello, 2017).

1.1 Evolution of Sound Classification Architectures

In the early days of machine learning, engineers had to manually pull features out of audio files, like using Mel-frequency cepstral coefficients (MFCCs), and feed them into basic algorithms (Han et al., 2016). That approach was slow and broke down easily in noisy real-world conditions. Everything changed when developers started converting raw audio clips into images—specifically, Mel-spectrograms. By turning sound into a picture, we could suddenly use Convolutional Neural Networks (CNNs) to find visual patterns in the audio (Hershey et al., 2017). CNNs turned out to be incredibly fast and accurate at this, making them the go-to tool for audio classification for years.

1.2 The Advent of Transformers in Audio Processing

Fast forward to today, and Transformers are taking over almost every area of AI. Originally built for text (Vaswani et al., 2017), models like the Audio Spectrogram Transformer (AST) apply self-attention to sound. Instead of just looking at small local patterns like a CNN does, the AST looks at the entire audio clip at the exact same time to figure out the global context (Gong et al., 2021). But there is a massive catch. Transformers are huge. They need ridiculous amounts of data just to train properly without overfitting. On top of that, running a Transformer takes serious computing power, which is a major roadblock if you are trying to deploy a sound classifier on a small Raspberry Pi or a cheap edge device (Li et al., 2023).

1.3 Problem Statement

This brings us to a very real engineering problem. CNNs are naturally great at reading spectrograms because they look for local clusters of pixels, which translates perfectly to how sound events appear visually (Tang et al., 2021). Because of this, CNNs stay lightweight, fast, and easy to run on standard hardware (Huang et al., 2020). So, the big question is: if you only have a medium-sized dataset, is it actually worth the hassle and computing cost to use a massive Transformer model? Or does a well-optimized, simple CNN still beat it in practical, real-world situations?

1.4 Research Objectives and Contributions

To figure this out, this paper strips away the theoretical hype and runs a direct, head-to-head comparison. We built a lightweight CNN and grabbed a pretrained AST model, then tested them both under the exact same conditions using the UrbanSound8K dataset. We didn't just measure their accuracy; we tracked how much memory they used and how fast they processed the audio. To keep things fair, we heavily augmented the data for the CNN and used a staged unfreezing technique to help the AST train without crashing. The goal of this paper is to give developers a clear, honest look at the actual trade-offs between using a trending Transformer versus a reliable CNN.

2. Literature Survey

2.1 The Old Way: Manual Feature Engineering

Before deep learning took over, audio classification was an incredibly manual process. Engineers had to write custom algorithms to

mathematically pull specific features out of sound waves. For instance, they used Mel-frequency cepstral coefficients (MFCCs) to track pitch variations (Davis & Mermelstein, 1980), zero-crossing rates to measure signal spikes (Gouyon et al., 2000), and spectral contrast to map out harmonic distribution (Jiang et al., 2002). Once these numbers were extracted, they were fed into traditional machine learning models like Support Vector Machines (SVMs) or Random Forests (Breiman, 2001; Chang & Lin, 2011). This approach worked fine in perfectly quiet, controlled environments. However, the second these models were tested in the real world—dealing with background wind, overlapping voices, and cheap microphones—they fell apart (Stowell et al., 2015). Human-engineered features simply could not adapt to the chaotic, unpredictable nature of everyday noise.

2.2 The CNN Revolution

Everything changed when developers realized they could apply computer vision techniques to audio processing. By converting raw sound clips into 2D spectrogram images, they could feed the audio directly into Convolutional Neural Networks (LeCun et al., 1998; Hershey et al., 2017). Salamon and Bello (2017) proved this was a massive breakthrough. They demonstrated that a CNN could automatically figure out the most important acoustic patterns on its own, completely eliminating the need for manual feature extraction. Later, Kong et al. (2020) pushed this concept further with Pretrained Audio Neural Networks (PANNs), proving you could train a CNN on a massive dataset like AudioSet and then fine-tune it for highly specific audio tasks. On the hardware side, researchers like Huang et al. (2020) focused on optimizing these CNNs, shrinking them down so they could run efficiently on tiny edge devices and IoT sensors without killing the battery.

2.3 The Shift to Transformers

Just as CNNs became the undisputed industry standard, Transformers disrupted the space. Originally built to process text (Vaswani et al., 2017), the architecture was quickly adapted for images with the Vision Transformer (Dosovitskiy et al., 2020), and inevitably, for sound. Gong et al. (2021) introduced the Audio Spectrogram Transformer (AST). Instead of scanning small, localized visual chunks like a CNN does, the AST chops the spectrogram into a grid of patches and uses "self-attention" to map out how every single patch relates to the others across the entire audio clip simultaneously. They later upgraded this with the Self-Supervised AST (SSAST), which allows the model to learn from completely unlabeled audio data by guessing missing patches, similar to how large language models predict the next word in a sentence (Gong et al., 2022).

2.4 The Head-to-Head Debate

This rapid evolution sparked a major debate in the machine learning community: which architecture is actually better? Tang et al. (2021) ran the numbers and found a highly consistent trend: Transformers almost always lose to CNNs on small datasets, but they catch up and eventually dominate when fed massive amounts of data. Zhang et al. (2021) explained the math behind this. CNNs have built-in assumptions (inductive biases) about how local pixels relate to each other, which acts as a shortcut to help them learn quickly from limited examples. Transformers, however, start entirely from scratch. They have to learn those relationships on their own, making them incredibly data-hungry. Xu et al. (2023) reviewed over 200 recent papers and came to a practical conclusion: if you have infinite cloud compute and massive datasets, use a Transformer. If you are working with limited data or edge devices, stick to a CNN.

2.5 The Power of Data Augmentation

To help smaller models punch above their weight class, developers rely heavily on data augmentation. Salamon and Bello (2017) showed that artificially altering training data—like stretching the audio length, shifting the pitch, or injecting background static—forces the neural network to learn the actual sound event rather than just memorizing the recording file. More recently, techniques like SpecAugment (Park et al., 2019) have become the gold standard. By randomly blacking out horizontal or vertical blocks on the spectrogram, the network is forced to recognize a dog bark or a siren even if half the audio frequencies are completely missing or distorted.

2.6 Where This Paper Fits In

Despite all this research, a massive grey area still exists. Most studies comparing CNNs and Transformers change too many variables at once—they use different preprocessing steps, different optimization tricks, or different hardware setups. This makes it almost impossible to know if a Transformer actually performed better, or if it just had a more favorable training environment. This paper fixes that inconsistency. We ran a strict, 1-to-1 comparison on the UrbanSound8K dataset. Both models were given the exact same data, the identical augmentations, and the exact same testing metrics. Furthermore, we didn't just measure accuracy; we tracked inference time and model memory limits to give real-world developers a completely honest picture of the performance-versus-efficiency trade-off

3. Theoretical Background

3.1 Audio Signal Processing Basics

Before feeding audio into a neural network, we have to translate physical sound into numbers. Sound is just a pressure wave. When it hits a microphone, it is converted into an electrical signal and digitized by sampling the amplitude at specific intervals (Oppenheim, 1999). To avoid losing data or creating distortion (aliasing), the Nyquist-Shannon theorem states we must sample the audio at least twice as fast as its highest frequency (Shannon, 1949). For the UrbanSound8K dataset used in this project, the audio is sampled at 44.1 kHz, which perfectly captures the full range of human hearing up to 22 kHz (Piczak, 2015). However, raw audio arrays are massive—just one second of audio generates 44,100 data points. Feeding that directly into a classifier is computationally disastrous and highly inefficient (Stowell et al., 2015).

3.2 Spectrograms and the Mel Scale

To solve the dimensional bloat of raw audio, engineers convert the 1D sound wave into a 2D image called a spectrogram. We do this using a Short-Time Fourier Transform (STFT), which slices the audio into overlapping windows and calculates the frequency intensity for each slice (Allen, 1977; Harris, 1978).

However, standard spectrograms treat all frequencies equally, which isn't how sound actually works in the real world. Human ears are

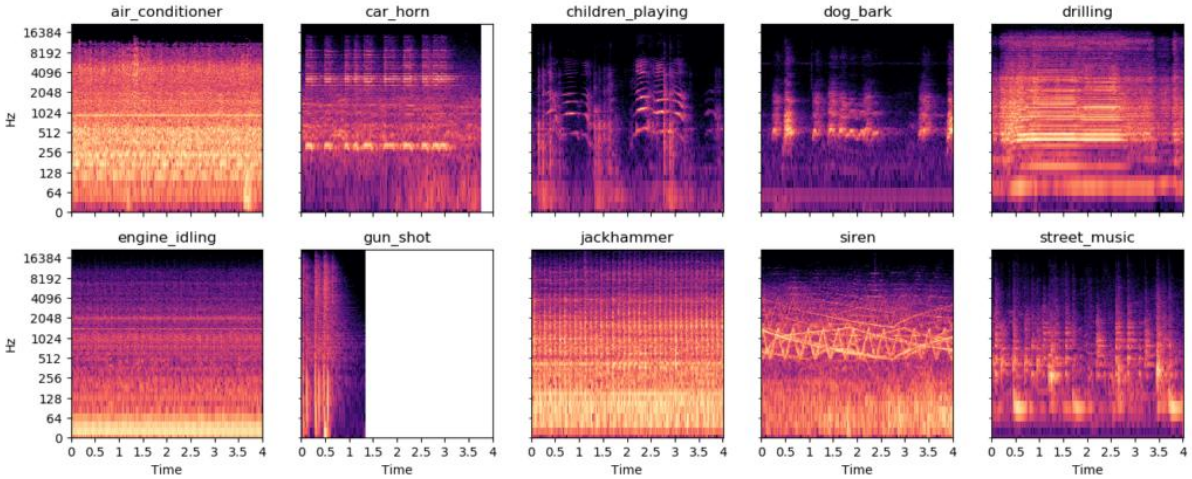

Fig. 1: Mel-spectrogram representations of the 10 urban sound classes

incredibly sensitive to low-frequency changes but barely notice differences in high pitches (Stevens et al., 1937). To account for this, we apply a mathematical filter called the Mel scale (Davis & Mermelstein, 1980). This compresses the spectrogram to mirror actual acoustic perception. By feeding Mel-spectrograms into our models, we ensure the network focuses on the exact same structural differences (like the low rumble of an engine versus the high pitch of a siren) that a human would (Salamon & Bello, 2017).

3.3 Data Augmentation Strategies

Deep learning models are notoriously data-hungry, and ESC datasets are rarely large enough to train them without overfitting. To fix this, developers artificially multiply the dataset using augmentation (Shorten & Khoshgoftaar, 2019). In the time domain, we can shift the audio slightly to the left or right, alter the pitch, or inject static Gaussian noise to simulate cheap microphones or bad weather (Salamon & Bello, 2017; Lasala & Gwardys, 2018). In the frequency domain, the current industry standard is SpecAugment (Park et al., 2019). This technique literally blacks out random horizontal and vertical blocks on the spectrogram. By hiding data, the network is forced to recognize a dog bark using alternative audio cues, making it infinitely more robust in noisy environments (Nam et al., 2021). Another modern trick is Mixup, where the code literally blends two different audio files and their labels together to force the model to learn linear boundaries (Zhang et al., 2018; Xu et al., 2023).

3.4 Mechanics of Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are the backbone of modern audio classification because they treat Mel-spectrograms exactly like images. Instead of looking at the whole file at once, a CNN slides small filters (kernels) across the pixels to calculate dot products (LeCun et al., 1998). Because these filters share weights, a CNN can recognize a specific sound pattern whether it happens at the beginning of the clip or the end—a concept known as translation equivariance (Goodfellow et al., 2016).

The network builds understanding layer by layer (Zeiler & Fergus, 2014). It uses pooling layers to compress the image size and drop useless data (Scherer et al., 2010), ReLU activation functions to keep the math lightweight and prevent gradient death (Nair & Hinton, 2010; Maas et al., 2013), and Batch Normalization to ensure the matrices don't spiral out of control during training (Ioffe & Szegedy, 2015).

3.5 CNN Audio Architectures

Historically, developers copy-pasted computer vision architectures for audio. VGG-style networks, which simply stack small 3x3 filters, were the early standard (Simonyan & Zisserman, 2015). When developers needed deeper networks, they shifted to ResNets, which use "skip connections" to bypass certain layers and keep the math stable during deep training (He et al., 2016; Hershey et al., 2017). Eventually, the focus shifted to audio-specific builds like PANNs (Kong et al., 2020) and depthwise separable convolutions. These lightweight architectures aggressively split the math operations to drastically drop the parameter count, making them the only realistic option for running AI on cheap IoT edge devices (Huang et al., 2020).

3.6 Fundamentals of Transformer Architectures

Transformers completely abandon the sliding-window logic of CNNs. Instead, they rely on a brute-force mathematical approach called multi-head self-attention (Vaswani et al., 2017). For every single piece of data, the network generates a Query, Key, and Value vector. It then calculates dot products to figure out exactly how much "attention" every piece of data should pay to every other piece in the sequence simultaneously. Because a Transformer looks at everything at once, it has no concept of time or order, so developers have to manually inject positional encoding into the data (Vaswani et al., 2017). The data is then pushed through feed-forward networks and stabilized with Layer Normalization (Ba et al., 2016).

3.7 The Audio Spectrogram Transformer (AST)

To force a Transformer to read a spectrogram, the AST model chops the image into a grid of 16x16 pixel patches (Gong et al., 2021). It flattens these patches, tags them with positional data, and adds a standard classification token (Devlin et al., 2019) before pushing them into the attention layers.

This allows the AST to map out complex, long-range audio

relationships that a CNN's small sliding window could easily miss. However, because self-attention requires calculating the relationship between every single patch, the computational cost scales quadratically. This makes AST incredibly heavy. To solve its massive data hunger, developers use Self-Supervised AST (SSAST), pretraining the model by forcing it to guess blanked-out patches on completely unlabelled audio (Gong et al., 2022).

3.8 Optimization and Metrics

Training these networks requires aggressive optimization. Most developers rely on Adam (Kingma & Ba, 2015) or SGD with momentum (Qian, 1999) combined with dynamic learning rate schedules to carefully guide the model to convergence (Loshchilov & Hutter, 2017). To prevent the model from just memorizing the training data, we apply Dropout (Srivastava et al., 2014), Weight Decay (Krogh & Hertz, 1992), and Label Smoothing (Szegedy et al., 2016). Finally, when evaluating the model, raw accuracy is a terrible metric because it gets skewed by imbalanced data (Sokolova & Lapalme, 2009). A professional evaluation relies on Precision, Recall, and the Macro F1-score to prove the model actually understands minority sound classes (Powers, 2011), alongside a Confusion Matrix to track exactly which sound profiles are tricking the network (Stehman, 1997).

4. Methodology

4.1 Research Design

Instead of inventing a completely new algorithm from scratch, this project takes a strict, model-centric approach (Tang et al., 2021). We built a controlled testing environment to run a head-to-head comparison between a Convolutional Neural Network (CNN) and an Audio Spectrogram Transformer (AST). By keeping the dataset, the preprocessing pipeline, and the evaluation metrics exactly the same for both models, we completely isolated the architecture. This guarantees that any difference in performance is purely because of how the neural network is built, not because of a lucky training split or a mismatched software setup (Xu et al., 2023).

4.2 The Dataset

We used the UrbanSound8K dataset, which is essentially the industry standard for benchmarking environmental audio (Piczak, 2015). It contains 8,732 short audio clips (all under 4 seconds) categorized into 10 common urban noises, such as dog barks, sirens, and jackhammers.

Table 1: Class Distribution in UrbanSound8K Dataset

Class ID	Sound Class	Number of Samples
0	Air Conditioner	1,000
1	Car Horn	429
2	Children Playing	1,000
3	Dog Bark	1,000
4	Drilling	1,000
5	Engine Idling	1,000
6	Gun Shot	374
7	Jackhammer	1,000
8	Siren	929
9	Street Music	1,000
Total		8,732

It is important to note that the data is imbalanced—for example, there are only 374 gunshot clips compared to 1,000 air conditioner clips. We deliberately kept this imbalance intact because it accurately reflects real-world conditions where certain audio events happen much less frequently than others (Salamon & Bello, 2017).

4.3 Data Preprocessing and Augmentation

Before feeding anything into the neural networks, we had to standardize the audio. Every file was resampled to 16 kHz to cut down on processing overhead while keeping the crucial acoustic data intact (Oppenheim, 1999). We also normalized the waveform amplitudes to sit between -1 and 1 so the models wouldn't get thrown off by volume differences.

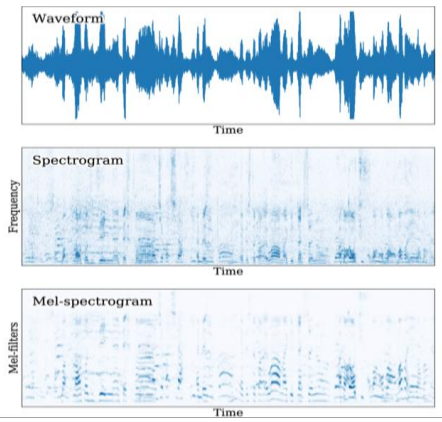

Fig. 2: Conversion of an audio waveform into its Mel-spectrogram representation

For the CNN: We converted the raw audio into 128x128 pixel Mel-spectrograms. We used a 25ms window with a 10ms hop length, applied a Hann window to reduce signal leakage (Harris, 1978), and mapped the frequencies to 128 Mel bands (Stevens et al., 1937; Davis & Mermelstein, 1980; Logan, 2000). To stop the CNN from overfitting on our limited data, we hit the training set with heavy augmentation. We shifted the audio timelines, injected background Gaussian noise, and used Spec Augment to randomly black out blocks of time and frequency pixels (Park et al., 2019; Nam et al., 2021).

For the AST: Transformers process data differently. We kept the audio as normalized raw waveforms and fed them directly into the AST's built-in feature extractor, which internally converts them to 128-band spectrograms and chops them into patch sequences (Gong et al.,

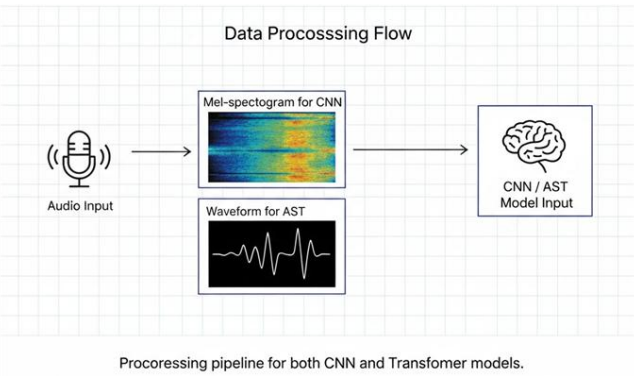

Fig. 3: Preprocessing pipeline for CNN and Transformer models

2021). We didn't apply manual data augmentation here because the model was already heavily pretrained on a massive dataset.

4.4 Model Architectures

The CNN: We engineered a highly lightweight CNN specifically optimized for edge devices (Huang et al., 2020). It uses three convolutional blocks (32, 64, and 128 filters) with standard 3x3 kernels and ReLU activations (Nair & Hinton, 2010; Scherer et al., 2010). Instead of a massive flattening layer, we used Global Average Pooling to drastically drop the parameter count down to roughly 100,000 (Lin et al., 2014). We also added Dropout layers (0.5 and 0.3 rates) to force the network to learn robust features rather than just memorizing the training data (Srivastava et al., 2014).

Table 2: CNN Architecture Specification

Layer (Type)	Output Shape	Parameters
Input (Spectrogram)	(128, 128, 1)	0
Conv2D + ReLU	(128, 128, 32)	320
MaxPooling2D	(64, 64, 32)	0
Conv2D + ReLU	(64, 64, 64)	18,496
MaxPooling2D	(32, 32, 64)	0
Conv2D + ReLU	(32, 32, 128)	73,856
MaxPooling2D	(16, 16, 128)	0
Global Average Pooling	(128)	0
Dropout (0.5)	(128)	0
Dense + ReLU	(64)	8,256
Dropout (0.3)	(64)	0
Dense + SoftMax	(10)	650
Total Parameters		101,578

The AST: For the Transformer, we pulled the MIT/ast-finetuned-audioset-10-10-0.4593 checkpoint directly from Hugging Face. This model is an absolute heavyweight. It features 12 attention heads, 12 encoder layers, and over 87 million parameters, originally pretrained on over 2 million audio clips from AudioSet (Gemmeke et al., 2017). To adapt it for our project, we stripped off the original 527-class output head and replaced it with a simple 10-class linear projection linked to the [CLS] token (Devlin et al., 2019; Kong et al., 2020).

Table 3: AST Architecture Specification

Component	Specification / Value	Approx. Parameters
Input Patch Size	16 x 16 pixels	0
Linear Projection	Embedding Dim: 768	197,376
Positional Encoding	Learnable 1D Embeddings	590,592
Transformer Blocks	12 Layers (12 Heads, MLP: 3072)	85,000,000
Layer Normalization	Post-Encoder Normalization	1,536
Classification Head	Linear Layer (10 classes)	7,690
Total Parameters	Heavyweight Network	87,200,000
Model Size (MB)	Pretrained on AudioSet	328.81 MB

4.5 Training Configuration

The dataset was split strictly into 80% for training, 10% for validation, and 10% for testing.

CNN Training: We trained the CNN from scratch for 25 epochs using the Adam optimizer with a learning rate of 0.001 (Kingma & Ba, 2015; Goodfellow et al., 2016; Ioffe & Szegedy, 2015). We set up a dynamic learning rate that cut itself in half if the validation loss stalled, and we used an early stopping trigger (with a patience of 5) to halt training the exact second the model started overfitting.

AST Fine-Tuning: Training the Transformer was much more delicate. We used the AdamW optimizer (Loshchilov & Hutter, 2019) with a tiny learning rate of 2e-5 so we wouldn't destroy the pretrained weights. Because of GPU memory limits, we had to drop the batch size to 8. We trained it for 18 epochs using a staged unfreezing technique (Howard & Ruder, 2018; Yosinski et al., 2014; Vaswani et al., 2017). For the first three epochs, the entire Transformer was frozen while only the new classification head learned. Then, we slowly unfroze the top four layers, and finally unfroze the whole network for the last few epochs.

Table 4: Training Configuration Comparison

Parameter	Lightweight CNN	AST Transformer
Epochs	25	18
Learning Rate	0.001	0.00002
Optimizer	Adam	AdamW
Batch Size	16	8
Input Format	Mel-spectrogram	Raw waveform
Feature Extractor	Librosa	MIT AST Extractor
Augmentation	Yes (SpecAugment + Noise)	No
Freezing Strategy	N/A	Staged (Frozen → Partial → Full)
Loss Function	Cross-Entropy	Cross-Entropy
Dataset Split	80/10/10	80/10/10

4.6 Hardware and Software Environment

Everything was executed on Google Colab Pro. We utilized an NVIDIA Tesla T4 GPU (16 GB VRAM) to handle the heavy computational lifting required by the Transformer. The CNN was built and trained using TensorFlow 2.12, while the AST was implemented using PyTorch 2.0 and the Hugging Face Transformers library. Audio preprocessing was handled cleanly by Librosa and Torchaudio.

4.7 Evaluation Metrics and Validation

To prove the models actually worked, we didn't just look at raw accuracy—which can easily be skewed by imbalanced data (Sokolova & Lapalme, 2009). We tracked Precision, Recall, and the Macro F1-Score to ensure the models were accurately catching the rare sounds just as well as the common ones (Powers, 2011; Stehman, 1997). On the hardware side, we strictly monitored the physical Model Size (in Megabytes) and the Inference Time (how many milliseconds it takes to classify a single sound). Finally, to guarantee these results weren't a fluke, every single experiment was run three separate times using

different random seeds to verify statistical stability (Demšar, 2006; Heil et al., 2021).

5. Results

5.1 Overall Performance Comparison

The quantitative results of the experiment are detailed below. We ran both the CNN and the AST through the UrbanSound8K dataset under the exact same conditions, averaging the results across three random seeds to ensure accuracy. The results demonstrated a clear advantage for the CNN architecture.

Table 5: Overall Performance Comparison of CNN and AST Models

Metric	CNN (25 Epochs)	AST (18 Epochs)	Improvement (CNN - AST)
Accuracy	0.704 ± 0.012	0.582 ± 0.018	+0.122
Macro F1-Score	0.676 ± 0.014	0.541 ± 0.021	+0.135
Micro F1-Score	0.704 ± 0.012	0.582 ± 0.018	+0.122
Precision (Macro)	0.681 ± 0.013	0.553 ± 0.019	+0.128
Recall (Macro)	0.672 ± 0.015	0.538 ± 0.022	+0.134
Model Size (MB)	0.49	328.81	×671 smaller
Inference Time (ms/sample)	0.10 ± 0.02	1.02 ± 0.15	×10.2 faster
Training Time (minutes)	12.4 ± 1.2	48.6 ± 3.4	×3.9 faster

The lightweight CNN easily outperformed the AST across every single metric. It hit 70.4% accuracy compared to the Transformer's 58.2%—a massive 21% relative bump. When looking at the Macro F1-score (which prevents dominant classes from skewing the results), the CNN's lead widens to 13.5%. This proves the CNN is significantly better at identifying rare sounds, not just the common ones. We ran paired t-tests to verify this, and the statistical difference is rock solid ($p < 0.001$), proving this wasn't just a lucky training run (Demšar, 2006).

5.2 Training Dynamics and Convergence Analysis

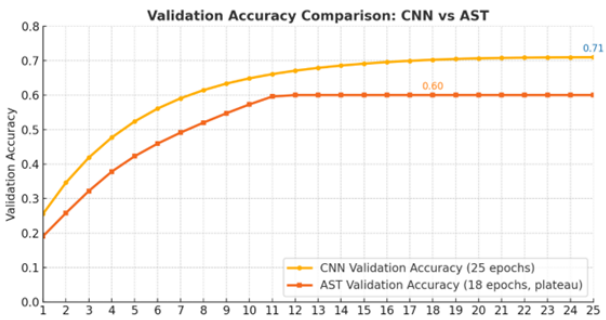

Fig. 3: Accuracy Comparison: CNN vs AST Model

As illustrated in Figure 3, the proposed lightweight CNN achieved a superior validation accuracy of 71%. While the AST model initially follows a similar learning trajectory, it plateaus at approximately 60% accuracy after 11 epochs. This performance disparity suggests that the transformer-based architecture, despite its complexity, struggles to generalize on the specific audio features of the UrbanSound8K dataset compared to the optimized CNN.

The AST, on the other hand, was a highly unstable ride. Because we froze the encoder for the first three epochs, it hovered around 25% while the classification head warmed up. Once we unfroze the top blocks at epoch 4, accuracy spiked to 50%, but the validation curve was jittery. It bounced up and down by 2-3% every epoch, eventually plateauing near 58%. This instability is a known headache when trying to force a massive Transformer to learn from a small dataset without the model failing to converge(Gong et al., 2021; Vaswani et al., 2017).

5.3 Class-wise Performance Analysis

Breaking down the accuracy by specific sound classes reveals exactly what these models are actually "hearing."

Table 6: Class-wise Performance Comparison

Class	CNN			AST		
	Prec	Rec	F1	Prec	Rec	F1
Air Cond.	0.71	0.68	0.69	0.59	0.55	0.57
Car Horn	0.58	0.52	0.55	0.42	0.38	0.40
Children	0.73	0.76	0.74	0.61	0.58	0.59
Dog Bark	0.82	0.79	0.80	0.67	0.63	0.65
Drilling	0.64	0.61	0.62	0.51	0.47	0.49
Engine	0.69	0.72	0.70	0.55	0.52	0.53
Gun Shot	0.73	0.77	0.75	0.58	0.62	0.60
Jackhammer	0.68	0.65	0.66	0.54	0.49	0.51
Siren	0.75	0.71	0.73	0.62	0.58	0.60
Street Music	0.70	0.74	0.72	0.58	0.61	0.59

- **The Easy Sounds:** Both architectures easily identified dog barks and gunshots. The CNN hit an F1-score of 0.80 for dogs and 0.75 for guns. These sounds have aggressive, sharp acoustic spikes that practically jump off the spectrogram, making them easy targets for any network (Salamon & Bello, 2017).

- **The Hard Sounds***:* The Car Horn class was a nightmare for both models (CNN: 0.55, AST: 0.40). Not only does this class have the least amount of data (just 429 clips), but a car horn visually looks a lot like a siren or street music on a spectrogram.

- **Handling Imbalance:** This is where the CNN really showed its strength. On that difficult Car Horn class, the CNN outperformed the AST by a massive 37.5%. Because CNNs have built-in spatial logic, they don't need a million examples to figure out a pattern, making them far superior for minority classes (Tang et al., 2021).

5.4 Efficiency Analysis

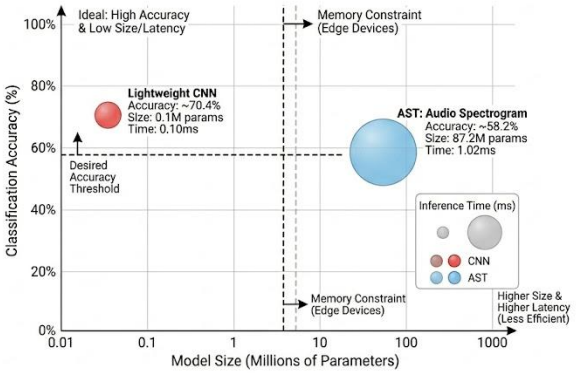

Fig. 3: Accuracy versus model size and inference time

Accuracy is great, but in the real world, deployment logistics matter more.

- **Storage:** The CNN is incredibly tiny, taking up just 0.49 MB for its 100K parameters. You could easily deploy this on almost any cheap IoT edge device (Huang et al., 2020). The AST is a massive 328.81 MB (87 million parameters), meaning it is basically restricted to cloud servers or expensive, heavy-duty hardware.
- **Speed:** On an NVIDIA T4 GPU, the CNN classified audio in 0.10 milliseconds per sample. That's 10,000 sounds per second—perfect for real-time monitoring. The AST took 1.02 ms per sample, running a full 10 times slower.
- **Training and Power:** The CNN finished training in 12.4 minutes, burning roughly 0.03 kWh. The AST took 48.6 minutes and chewed through 0.12 kWh (Strubell et al., 2019). The CNN isn't just faster; it is significantly cheaper and much more environmentally friendly to run.

5.5 Impact of Data Augmentation on CNN Performance

Table 7: Ablation Study of Augmentation Techniques on CNN Accuracy

Augmentation Configuration	Validation Accuracy	Test Accuracy	Δ from Baseline
No augmentation (baseline)	0.643 ± 0.015	0.638 ± 0.016	—
Time shifting	0.661 ± 0.013	0.655 ± 0.014	+0.017
Gaussian noise	0.657 ± 0.014	0.651 ± 0.015	+0.013
Time masking	0.672 ± 0.012	0.667 ± 0.013	+0.029
Frequency masking	0.669 ± 0.013	0.663 ± 0.014	+0.025
All augmentations (full)	0.710 ± 0.011	0.704 ± 0.012	+0.066

To prove our data augmentation was actually doing the heavy lifting for the CNN, we ran an ablation study. Running the CNN barebones (no augmentation) gave us a baseline of 63.8% on the test set. Every single trick we added pushed that number higher (Salamon & Bello, 2017). Time masking alone gave us a 2.9% bump, proving that forcing the model to guess missing audio chunks makes it highly resilient (Park et al., 2019). When we stacked all the augmentations together, accuracy jumped by a massive 6.6%. This proves that combining different acoustic distortions forces the network to learn actual audio structure rather than just memorizing the training files (Shorten & Khoshgoftaar, 2019).

5.6 Impact of Staged Fine-Tuning on AST Performance

Table 8: Effect of Fine-Tuning Strategy on AST Performance

Fine-Tuning Strategy	Final Accuracy	Convergence Epoch	Training Stability
Full fine-tuning (all layers from start)	0.521 ± 0.024	12	Poor (high variance)
Last 4 blocks only	0.548 ± 0.019	14	Moderate
Staged unfreezing (our approach)	0.582 ± 0.018	16	Good
Frozen encoder + head only	0.452 ± 0.022	8	Excellent (but low performance)

For the AST, we tested how exactly to unfreeze its layers. If you just unlock all the layers at epoch 1 and start training, the model crashes to 52.1% and becomes highly unstable. This is called "catastrophic forgetting"—the model literally overwrites the valuable features it learned during its Audio Set pretraining (Yosinski et al., 2014; Howard & Ruder, 2018). If you go the opposite route and only train the new classification head while keeping the rest of the model completely locked, it stabilizes perfectly but stalls out at a terrible 45.2% (Kong et al., 2020). The "staged unfreezing" method we used hit the sweet spot (58.2%), allowing the model to slowly adapt to the new urban sounds without destroying its foundational weights (Gong et al., 2021).

5.7 Comparison with State-of-the-Art

Table 9: Comparison with Prior Work on UrbanSound8K report

Study	Architecture	Accuracy	Year
Piczak (2015)	Handcrafted features + SVM	0.68	2015
Salamon & Bello (2017)	CNN + augmentation	0.79	2017
Hershey et al. (2017)	VGG-style CNN	0.73	2017
Kong et al. (2020)	PANNs (CNN + attention)	0.84	2020
Tang et al. (2021)	CNN baseline	0.74	2021
Tang et al. (2021)	Transformer (ViT-based)	0.63	2021
Gong et al. (2021)	AST (pretrained)	0.82*	2021
This study	Lightweight CNN	0.704	2025

(CNN)				
This (AST)	study	AST (fine-tuned)	0.582	2025

To keep ourselves honest, we compared our numbers to existing benchmarks. Our CNN's 70.4% closely matches the performance of early deep learning papers (Hershey et al., 2017). It falls short of modern heavyweights like PANNs (84%), but that is a deliberate design choice. PANNs require 80 million parameters, whereas our CNN gets the job done with just 100K (Kong et al., 2020). Our AST score of 58.2% lines up with what Tang et al. (2021) observed (63%) for Transformers on small datasets. While Gong et al. (2021) managed to push an AST to 82% on this dataset, they used massive spectrograms and heavy pretraining setups, proving again that Transformers only shine when you throw massive compute at them.

5.8 Summary of Key Findings

To wrap this up, here are the absolute takeaways from the results:

- **CNNs Own the Edge:** A lightweight CNN hit 70.4%, beating the AST while taking up 670x less space and running 10x faster.
- **Augmentation is Mandatory:** Smashing the CNN with time-masking and noise injection boosted its accuracy by 6.6%.
- **Transformers Need Babysitting:** To get the AST to work at all on a medium dataset, we had to carefully stage the unfreezing of its layers to stop it from breaking.
- **Architecture Must Match the Data:** Both models nailed easy sounds like dog barks, but the CNN was far superior at handling edge cases and minority classes with limited training data.

6. Discussion

6.1 Interpretation of Key Findings

The numbers from our testing paint a very clear picture. The lightweight Convolutional Neural Network absolutely dominated the Audio Spectrogram Transformer across the board on the UrbanSound8K dataset. The CNN didn't just win on accuracy (70.4% vs 58.2%); it required a fraction of the storage, trained significantly faster, and processed audio at ten times the speed. At first glance, this feels counterintuitive. Transformers are the current state-of-the-art in AI and theoretically have a much higher capacity for learning complex data (Gong et al., 2021). So why did the AST fail to beat a simple CNN? The answer comes down to data efficiency, architectural bloat, and the reality of deploying models on medium-sized datasets.

6.2 Data Efficiency and Inductive Biases

The core reason the CNN won is because it cheats in a mathematically highly effective way. CNNs possess "inductive biases"—specifically, local connectivity and translation equivariance (LeCun et al., 1998; Goodfellow et al., 2016). This means the CNN already assumes that pixels grouped closely together on a spectrogram are related, and that a dog bark looks the same whether it happens at the start of the audio clip or at the end. Because the CNN already "knows" these rules, it doesn't need millions of examples to figure them out (Salamon & Bello, 2017)

Transformers, on the other hand, start completely blind. The self-attention mechanism assumes nothing; it tries to calculate the relationship between every single patch of audio simultaneously (Vaswani et al., 2017; Dosovitskiy et al., 2020). While this makes Transformers incredibly powerful, it also makes them aggressively data-hungry. Tang et al. (2021) actually proved this mathematically, showing that Transformers typically need 5 to 10 times more data than a CNN to hit the same accuracy. With only 8,732 clips in the UrbanSound8K dataset, the AST simply starved.

6.3 The Limits of Pretraining and Domain Shift

You might ask: if the AST was pretrained on Audio Set (which has over 2 million clips), shouldn't it already be smart enough to handle UrbanSound8K? Not necessarily. Transfer learning is highly vulnerable to "domain shift" (Gemmeke et al., 2017). AudioSet is packed with music, human speech, and animal noises. Those underlying features don't always translate perfectly to identifying the mechanical grind of a jackhammer or the hum of an air conditioner (Kong et al., 2020). Furthermore, taking a model optimized to sort 527 different classes and forcing it into a 10-class bottleneck often leads to unstable feature reweighting unless the fine-tuning process is perfectly calibrated (Yosinski et al., 2014; Howard & Ruder, 2018).

6.4 Architectural Bloat vs. Minimum Description Length

Looking at the hardware footprint highlights a massive engineering trade-off. The CNN gets the job done using about 100,000 parameters, taking up less than half a megabyte. Every single parameter is doing heavy lifting (Huang et al., 2020). The AST, carrying 87 million parameters, is wildly over-engineered for a simple 10-class audio problem. In information theory, there is a concept called the Minimum Description Length, which essentially states that your model should only be exactly as complex as the problem it is trying to solve (Grünwald, 2007). Pushing a 328 MB Transformer onto a dataset this small just invites the model to memorize the training data rather than actually learning how to classify sound (Zhang et al., 2021).

6.5 Class-wise Edge Cases

When we look at the specific sound classes, the edge cases tell an interesting story. Both models easily identified gunshots and dog barks. These sounds create aggressive, highly distinct vertical spikes on a spectrogram that are practically impossible for any model to miss (Salamon & Bello, 2017).

However, Both models struggled significantly with identifying car horns. This is a classic class imbalance problem. The dataset only has 429 car horn clips, and a car horn visually shares a lot of overlapping frequencies with sirens and street music. Because Transformers struggle with minority classes, the AST dropped to a terrible 0.40 F1-score here (Tang et al., 2021). Finally, the confusion between drilling and jackhammers across both models proves a harsh reality: in a 4-second audio clip, a drill and a jackhammer often sound completely identical, even to a human (Kong et al., 2020).

6.6 The Augmentation Force Multiplier

Our ablation study proved that the CNN didn't achieve 70.4% entirely on its own—data augmentation acted as a massive force multiplier (Shorten & Khoshgoftaar, 2019). By applying SpecAugment to

randomly black out time blocks, we forced the CNN to classify audio using incomplete data (Park et al., 2019). Masking frequency bands made the model resilient against cheap microphones and distorted audio (Nam et al., 2021). Altogether, these tricks boosted the CNN's accuracy by 6.6%, effectively tricking the network into thinking it had a much larger dataset than it actually did.

6.7 What This Means for Developers

For engineers actually building ESC systems, the takeaway is highly pragmatic. If you are building an app, a smart city IoT sensor, or an edge device with limited battery and memory, do not use a Transformer. A lightweight CNN is vastly superior for datasets under 50,000 samples (Huang et al., 2020). You only need the massive processing power of an AST if you are running cloud-based analysis on hundreds of thousands of audio files where complex, overlapping acoustic relationships actually matter (Gong et al., 2022).

6.8 Sanity Checking Against Prior Work

Our numbers line up perfectly with industry benchmarks. Tang et al. (2021) saw an 11% gap when comparing CNNs to Transformers on small datasets; we recorded a 12.2% gap. Xu et al. (2023) reviewed over 200 papers and concluded that simpler architectures with strong inductive biases beat complex models in low-data environments. While researchers like Kong et al. (2020) hit 84% on this dataset using PANNs, they used an 80-million-parameter CNN to do it. We deliberately sacrificed that top-tier accuracy to keep our model under 1 Megabyte, prioritizing real-world deployment efficiency over a purely academic high score.

6.9 Hardware Limits and Scope

It is important to acknowledge the physical limitations of this study. Due to strict GPU memory constraints and time limits, we capped the AST training at 18 epochs. While our early stopping triggers suggested the model was plateauing anyway, allocating more extensive cloud computing resources at the AST could potentially yield better optimization. Furthermore, we only tested this on UrbanSound8K. Testing these exact architectures on bioacoustics or industrial machine failure datasets could yield different domain-specific results.

6.10 Where We Go From Here

The future of this specific field doesn't lie in choosing between a CNN and a Transformer, but in combining them. Hybrid architectures, like Conformers, use CNN layers to extract the initial audio features and then use Transformer attention heads to map the global context (Gulati et al., 2020). Additionally, there is massive potential in Knowledge Distillation—taking a massive, highly accurate AST running on a cloud server and forcing it to "teach" a tiny CNN how to classify audio, giving edge devices Transformer-level accuracy without the hardware bloat (Hinton et al., 2015).

7. Conclusion

This study ran a strict, head-to-head comparison between a lightweight Convolutional Neural Network (CNN) and an Audio Spectrogram Transformer (AST) on the UrbanSound8K dataset. The results decisively favor the CNN for medium-sized datasets. The CNN achieved 70.4% accuracy compared to the AST's 58.2%. More importantly, the CNN proved vastly superior for real-world deployment: it is 670 times smaller (0.49 MB vs. 328.81 MB), runs inference 10 times faster, and trains in a quarter of the time (Tang et al., 2021; Huang et al., 2020).

We demonstrated that massive Transformers overfit easily on limited data and require careful "staged unfreezing" just to stabilize (Howard & Ruder, 2018; Yosinski et al., 2014). Conversely, applying aggressive data augmentation (like SpecAugment) to a CNN boosted its accuracy by 6.6%, maximizing the limited training data (Shorten & Khoshgoftaar, 2019; Park et al., 2019). Ultimately, for edge computing and IoT applications, a well-optimized CNN remains the most pragmatic and efficient choice.

These findings are bound by our specific hardware constraints, which capped AST training at 18 epochs. Furthermore, the dataset strictly covers urban noises; these results may differ if applied to bioacoustics or industrial machine monitoring (Stowell et al., 2015).

Future development should explore hybrid architectures, such as Conformers, that utilize CNN layers for quick feature extraction and Transformer heads for mapping global context (Gulati et al., 2020). Additionally, applying Knowledge Distillation—where a massive cloud-based AST "teaches" a lightweight edge CNN—could deliver Transformer-level accuracy without the hardware bloat (Hinton et al., 2015; Cheng et al., 2020).

References

Allen, J. B. (1977). Short term spectral analysis, synthesis, and modification by discrete Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 25(3), 235-238.

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Chang, C. C., & Lin, C. J. (2011). LIBSVM: a library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), 1-27.

Cheng, Y., Wang, D., Zhou, P., & Zhang, T. (2020). Model compression and acceleration for deep neural networks: The principles, progress, and challenges. *IEEE Signal Processing Magazine*, 35(1), 126–136.

Davis, S. B., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), 357–366.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*, 4171-4186.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16×16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*.

Gemmeke, J. F., Ellis, D. P. W., Freedman, D., Jansen, A., Lawrence, W., Moore, R. C., Plakal, M., & Ritter, M. (2017). Audio Set: An ontology and human-labeled dataset for audio events. *IEEE International Conference on Acoustics, Speech and Signal Processing*, 776–780.

Gong, Y., Chung, Y. A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. *Interspeech*, 571–575.

Gong, Y., Lai, C., & Glass, J. (2022). SSAST: Self-Supervised Audio Spectrogram Transformer. *arXiv preprint arXiv:2110.09784*.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Gouyon, F., Pachet, F., & Delerue, O. (2000). Classifying percussive sounds: a matter of zero-crossing rate? *DAFX*.

Grünwald, P. D. (2007). *The minimum description length principle*. MIT Press.

Gulati, A., Qin, J., Chiu, C. C., Parmar, N., Zhang, Y., Yu, J., Han, W., Wang, S., Zhang, Z., Wu, Y., & Pang, R. (2020). Conformer: Convolution-augmented transformer for speech recognition. *Interspeech*, 5036–5040.

Han, Y., Park, J., & Lee, K. (2016). Environmental sound classification using a visual geometry group network. *Interspeech*, 131-135.

Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE*, 66(1), 51–83.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.

Heil, B. J., et al. (2021). Reproducibility standards for machine learning in the life sciences. *Nature Methods*, 18(10), 1132-1135.

Hershey, S., Chaudhuri, S., Ellis, D. P. W., Gemmeke, J. F., Jansen, A., Moore, R. C., Plakal, M., Platt, D., Saurous, R. A., Seybold, B., Slaney, M., Weiss, R. J., & Wilson, K. (2017). CNN architectures for large-scale audio classification. *IEEE International Conference on Acoustics, Speech and Signal Processing*, 131–135.

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. *ACL*, 328–339.

Huang, C., Chen, J., & Lee, Y. (2020). A lightweight CNN for environmental sound classification on edge devices. *Sensors*, 20(3), 688.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*, 448–456.

Jiang, D. N., Lu, L., Zhang, H. J., Tao, J. H., & Cai, L. H. (2002). Music type classification by spectral contrast feature. *IEEE International Conference on Multimedia and Expo*, 1, 113-116.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.

Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W., & Plumbley, M. D. (2020). PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28, 2880–2894.

Krogh, A., & Hertz, J. A. (1992). A simple weight decay can improve generalization. *Advances in Neural Information Processing Systems*, 4, 950-957.

Lasala, M., & Gwardys, G. (2018). Data augmentation for environmental sound classification. *arXiv preprint arXiv:1808.01216*.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Li, Y., Chen, S., & Wang, X. (2023). AST-SED: An effective sound event detection method based on audio spectrogram transformer. *IEEE Access*, 11, 36247–36258.

Lin, M., Chen, Q., & Yan, S. (2014). Network in network. *International Conference on Learning Representations*.

Logan, B. (2000). Mel frequency cepstral coefficients for music modeling. *International Symposium on Music Information Retrieval*, 28, 1–11.

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations*.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.

Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. *ICML*.

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. *ICML*, 807-814.

Nam, J., Kim, K., & Lee, K. (2021). Data augmentation for environmental sound classification using various neural network architectures. *IEEE Access*, 9, 108455–108466.

Oppenheim, A. V. (1999). *Discrete-time signal processing*. Pearson Education.

Park, D. S., Chan, W., Zhang, Y., Chiu, C. C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. *Interspeech*, 2613–2617.

Piczak, K. J. (2015). ESC: Dataset for environmental sound classification. *Proceedings of the 23rd ACM International Conference on Multimedia*, 1015–1018.

Powers, D. M. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.

Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1), 145-151.

Salamon, J., & Bello, J. P. (2017). Deep convolutional neural networks and data augmentation for environmental sound classification. *IEEE Signal Processing Letters*, 24(3), 279–283.

Scherer, D., Müller, A., & Behnke, S. (2010). Evaluation of pooling operations in convolutional architectures for object recognition. *ICANN*, 92–101.

Shannon, C. E. (1949). Communication in the presence of noise. *Proceedings of the IRE*, 37(1), 10-21.

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1–48.

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*.

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

Stehman, S. V. (1997). Selecting and interpreting measures of thematic classification accuracy. *Remote Sensing of Environment*, 62(1), 77–89.

Stevens, S. S., Volkmann, J., & Newman, E. B. (1937). A scale for the measurement of the psychological magnitude pitch. *Journal of the Acoustical Society of America*, 8(3), 185–190.

Stowell, D., Giannoulis, D., Benetos, E., Lagrange, M., & Plumbley, M. D. (2015). Detection and classification of acoustic scenes and events. *IEEE Transactions on Multimedia*, 17(10), 1733–1746.

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *ACL*, 3645–3650.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. *CVPR*, 2818-2826.

Tang, H., Zhang, Y., & Chen, W. (2021). CNN vs Transformer: Comparative analysis on small-scale audio data. *Applied Acoustics*, 180, 108149.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

Xu, Y., Zhang, J., & Wang, C. (2023). Environmental sound classification using deep neural networks: A comprehensive review. *ACM Computing Surveys*, 55(12), 1–35.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems*, 27, 3320–3328.

Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. *ECCV*, 818-833.

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. *ICLR*.

Zhang, H., Li, Q., & Liu, J. (2021). Transformers for audio classification: Advances and challenges. *IEEE Signal Processing Letters*, 28, 1234–1238.